

```

import {
  MapContainer,
  TileLayer,
  Marker,
  useMapEvents,
  useMap,
} from "react-leaflet";
import { useEffect, useState } from "react";
import axios from "axios";
import toast from "react-hot-toast";
import { restaurantService } from "../main";
import L from "leaflet";
import { LuLocateFixed } from "react-icons/lu";
import { BiLoader, BiPlus, BiTrash } from "react-icons/bi";

// 🔑 Fix leaflet marker icon issue
delete (L.Icon.Default.prototype as any)._getIconUrl;
L.Icon.Default.mergeOptions({
  iconRetinaUrl:
    "https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon-2x.png",
  iconUrl: "https://unpkg.com/leaflet@1.9.4/dist/images/marker-
icon.png",
  shadowUrl: "https://unpkg.com/leaflet@1.9.4/dist/images/marker-
shadow.png",
});

interface Address {
  _id: string;
  formattedAddress: string;
  mobile: number;
}

// 📍 Click-to-select location
const LocationPicker = ({
  setLocation,
}: {
  setLocation: (lat: number, lng: number) => void;
}) => {
  useMapEvents({
    click(e) {
      setLocation(e.latlng.lat, e.latlng.lng);
    },
  },

```

```

    });
    return null;
  });

  // 📍 Locate me button
  const LocateMeButton = ({
    onLocate,
  }): {
    onLocate: (lat: number, lng: number) => void;
  }) => {
    const map = useMap();

    const locateUser = () => {
      if (!navigator.geolocation) {
        toast.error("Geolocation not supported");
        return;
      }

      navigator.geolocation.getCurrentPosition(
        (pos) => {
          const { latitude, longitude } = pos.coords;
          map.flyTo([latitude, longitude], 16, { animate: true });
          onLocate(latitude, longitude);
        },
        () => toast.error("Location permission denied")
      );
    };

    return (
      <button
        onClick={locateUser}
        className="absolute right-3 top-3 z-1000 flex items-center gap-2
rounded-lg bg-white px-3 py-2 text-sm shadow hover:bg-gray-100"
      >
        <LuLocateFixed size={16} />
        Use current location
      </button>
    );
  };

  const AddAddressPage = () => {
    const [addresses, setAddresses] = useState<Address[]>([]);

```

```

const [loading, setLoading] = useState(true);
const [adding, setAdding] = useState(false);
const [deletingId, setDeletingId] = useState<string | null>(null);

// 📋 Form state
const [mobile, setMobile] = useState("");
const [formattedAddress, setFormattedAddress] = useState("");
const [latitude, setLatitude] = useState<number | null>(null);
const [longitude, setLongitude] = useState<number | null>(null);

// 🌐 Reverse geocoding
const fetchFormattedAddress = async (lat: number, lng: number) => {
  try {
    const res = await fetch(
      `https://nominatim.openstreetmap.org/reverse?format=json&lat=${lat}&lon=${lng}`
    );
    const data = await res.json();
    setFormattedAddress(data.display_name || "");
  } catch {
    toast.error("Failed to fetch address");
  }
};

const setLocation = (lat: number, lng: number) => {
  setLatitude(lat);
  setLongitude(lng);
  fetchFormattedAddress(lat, lng);
};

// 📡 Fetch addresses
const fetchAddresses = async () => {
  try {
    const { data } = await
    axios.get(`${restaurantService}/api/address/all`, {
      headers: {
        Authorization: `Bearer ${localStorage.getItem("token")}`,
      },
    });
    setAddresses(data || []);
  } catch {
    toast.error("Failed to load addresses");
  }
};

```

```
    } finally {
      setLoading(false);
    }
  };

useEffect(() => {
  fetchAddresses();
}, []);

// + Add address
const addAddress = async () => {
  if (
    !mobile ||
    !formattedAddress ||
    latitude === null ||
    longitude === null
  ) {
    toast.error("Please select location on map");
    return;
  }

  try {
    setAdding(true);

    await axios.post(
      `${restaurantService}/api/address/new`,
      {
        formattedAddress,
        mobile,
        latitude,
        longitude,
      },
      {
        headers: {
          Authorization: `Bearer ${localStorage.getItem("token")}`,
        },
      }
    );

    toast.success("Address added");
    setMobile("");
    setFormattedAddress("");
  }
}
```

```

        setLatitude(null);
        setLongitude(null);
        fetchAddresses();
    } catch (error: any) {
        toast.error(error.response?.data?.message || "Failed");
    } finally {
        setAdding(false);
    }
};

// 🗑️ Delete address
const deleteAddress = async (id: string) => {
    if (!window.confirm("Delete this address?")) return;

    try {
        setDeletingId(id);

        await axios.delete(`${restaurantService}/api/address/${id}`, {
            headers: {
                Authorization: `Bearer ${localStorage.getItem("token")}`,
            },
        });

        toast.success("Address deleted");
        fetchAddresses();
    } catch {
        toast.error("Failed to delete address");
    } finally {
        setDeletingId(null);
    }
};

return (
    <div className="mx-auto max-w-4xl px-4 py-6 space-y-6">
        <h1 className="text-2xl font-bold">Select Delivery Address</h1>

        { /* 🗺️ Map */ }
        <div className="relative h-100 w-full overflow-hidden rounded-lg
border">
            <MapContainer
                center={[latitude || 28.6139, longitude || 77.209]}
                zoom={13}

```

```

        className="h-full w-full"
        style={{ height: "100%", width: "100%" }}
      >
      <TileLayer
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution='&copy; <a
href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>'
      />
      <LocationPicker setLocation={setLocation} />
      <LocateMeButton onLocate={setLocation} />
      {latitude && longitude && <Marker position={[latitude,
longitude]} />}
    </MapContainer>
  </div>

  {/* 📍 Selected address */}
  {formattedAddress && (
    <div className="rounded-lg border bg-green-50 p-3 text-sm">
      📍 {formattedAddress}
    </div>
  )}

  {/* 📱 Mobile */}
  <input
    type="number"
    placeholder="Mobile number"
    value={mobile}
    onChange={(e) => setMobile(e.target.value)}
    className="w-full rounded-lg border px-4 py-2"
  />

  {/* + Save */}
  <button
    disabled={adding}
    onClick={addAddress}
    className="flex items-center justify-center gap-2 rounded-lg
bg-[#E23744] px-4 py-3 text-white hover:bg-[#d32f3a] disabled:opacity-
50"
  >
    {adding ? <BiLoader className="animate-spin" /> : <BiPlus />}
    Save Address
  </button>

```



```
export default AddAddressPage;
```